BSc (Hons) in **Computer Science**

Faculty of Computing

SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Practical 01

> **Objective & Lecture Mapping:** Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the Git Hub Repo. Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

P – Performance Measure: The objective evaluation metric that grades environmental success.
E – Environment: The external world state, contexts, and systems our agent inhabits

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

self.width
self.height
self.walls
self.food_positions

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

The baseline environment is Multi-Agent because it includes the main agent and one or more opponent agents (self.opponents). These opponents move independently and interact with the main agent, making the environment a multi-agent system rather than a single-agent one.

Task 1.2: The Perception Subsystem (get_percept)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete history of all observations an agent has received from the environment since it started operating.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

It represents the Sensors (S) component of PEAS. get_percept() gathers information from the environment and returns it to the agent.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

Partially Observable. The agent does not receive the complete state of the environment.

Task 1.3: Action Execution (execute_action)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

The Performance Measure (P) is updated. The score is changed: self.score -= 5

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

Because the goal is to measure how successfully the agent achieves its objectives in the environment, not how much computation it performs.

Part 2: Guided Modification & Deep Theory Mapping

> Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (__init__)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the

environment but intentionally hide this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

The traps exist in the environment, but the agent has no sensor to detect them. Therefore, the agent has less information about the true environment state.

Step 2.2: Updating the Perception Subsystem (get_percept)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

'smells_toxin': tuple(self.agent_pos) in self.toxic_traps
This helps the agent make better decisions, such as avoiding toxic traps and choosing safer paths. Since a rational agent selects actions that maximize its expected performance measure, this new sensor helps it maximize its expected

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

In this scenario, the agent is rewarded simply for cleaning a square. Instead of keeping the environment permanently clean, the agent exploits the metric by cleaning a square, making it dirty again, and then cleaning it repeatedly. By repeating this cycle indefinitely, the agent continues to earn rewards without actually achieving the intended goal of maintaining a clean environment. This

Finalize and Lock Lab 01 Analytical Evaluation

FACULTY OF COMPUTING